Object Oriented Programming with Java

Lecture notes for Units 1 and 2: language basics, OOP, strings, exceptions, collections and multithreading.

1. Introduction to Java

Java is a high-level, class-based, object oriented programming language. It was designed to have as few implementation dependencies as possible, so that developers can write a program once and run it on any device that has a Java Virtual Machine (JVM).

1.1 Features of Java

- **Simple and familiar:** syntax similar to C and C++, without pointers or operator overloading.
- **Object oriented:** everything except the primitive types is an object.
- **Platform independent:** source code is compiled into bytecode, not machine code.
- **Robust:** strong type checking, automatic garbage collection and exception handling.
- **Secure:** no explicit pointers, and bytecode is verified before it runs.
- **Multithreaded:** threads are built into the language and the standard library.

1.2 How a Java program runs

The compiler, javac, translates a .java source file into a .class file that contains bytecode. The JVM does not understand Java source code; it executes bytecode, which is why the same .class file runs on every platform that has a JVM. Inside the JVM, the class loader loads classes, the bytecode verifier checks them, and the execution engine runs them. The Just-In-Time (JIT) compiler translates frequently executed bytecode into native machine code to improve performance.

1.3 JDK, JRE and JVM

> JDK = JRE + development tools (javac, java, javadoc, jdb). JRE = JVM + core class libraries.

Only the JVM is platform specific. A developer installs the JDK; a user who only runs programs needs the JRE.

1.4 JVM memory areas

The method area stores class-level data such as static variables and method code. The heap stores all objects and is shared by every thread. Each thread has its own stack, which holds a frame for every method call, including its local variables. Garbage collection reclaims heap memory used by objects that are no longer reachable from any live reference.

2. Data Types, Classes and Objects

2.1 Primitive data types

Type	Size	Range or values	Default
byte	8 bits	-128 to 127	0
short	16 bits	-32,768 to 32,767	0
int	32 bits	-2^31 to 2^31 - 1	0
long	64 bits	-2^63 to 2^63 - 1	0L
float	32 bits	about 7 decimal digits	0.0f
double	64 bits	about 15 decimal digits	0.0d
char	16 bits	Unicode characters	'\u0000'
boolean	JVM dependent	true or false	false

In general, an n-bit signed integer type can hold values from -2^(n-1) to 2^(n-1) - 1. Local variables are not given default values and must be initialised before use; instance variables get default values (0, false or null).

2.2 Classes and objects

A class is a blueprint that defines the fields (state) and methods (behaviour) of its objects. An object is an instance of a class, created at run time with the new keyword, which allocates memory on the heap and calls a constructor.

```
class Student {
    private String name;
    private int marks;

    Student(String name, int marks) {   // parameterised constructor
        this.name = name;
        this.marks = marks;
    }
}
Student s = new Student("Asha", 87);
```

2.3 Constructors

A constructor has the same name as its class and no return type, not even void. It runs once, when the object is created. If a class defines any constructor, the compiler no longer provides the default constructor. Constructors can be overloaded, and one constructor can call another with this(...), which must be the first statement.

2.4 The static keyword

A static field belongs to the class and is shared by all objects. A static method can be called without creating an object and cannot use this or access instance members directly. A static block runs once, when the class is loaded.

3. Object Oriented Programming Concepts

3.1 Encapsulation

Encapsulation binds data and the methods that operate on it into a single unit and hides the data from outside access. Fields are declared private and are read or changed only through public getter and setter methods, which can validate the values.

3.2 Inheritance

Inheritance lets a subclass acquire the fields and methods of a superclass using the extends keyword, which models an "is-a" relationship. The super keyword calls the superclass constructor or an overridden superclass method. Java does not support multiple inheritance of classes, but a class can implement any number of interfaces.

Types of inheritance supported with classes: single, multilevel and hierarchical.

3.3 Polymorphism

Polymorphism means one name, many forms. Method overloading defines several methods with the same name but different parameter lists in the same class. Method overriding redefines an inherited method in a subclass with the same signature. Overloading is resolved at compile time from the argument types, whereas overriding is resolved at run time by dynamic method dispatch.

```
Animal a = new Dog();   // superclass reference, subclass object
a.sound();              // calls Dog.sound() at run time
```

3.4 Abstraction

Abstraction shows only the essential features of an object and hides the implementation. An abstract class may contain abstract and concrete methods and cannot be instantiated. An interface declares methods that implementing classes must provide.

Abstract class	Interface
Can have abstract and concrete methods	Abstract methods, plus default and static methods since Java 8
Can have instance variables and constructors	Only public static final constants; no constructors
A class extends only one abstract class	A class can implement many interfaces
Members can have any access modifier	Methods are public by default

3.5 The final keyword

A final variable cannot be reassigned, a final method cannot be overridden and a final class cannot be extended.

4. Strings and StringBuilder

A String in Java is an object of the class java.lang.String that represents a sequence of characters. String objects are immutable: every method that appears to modify a String returns a new String object. Immutability makes strings safe to share between threads and allows the JVM to cache them.

4.1 The string constant pool

String literals are stored in a special area of the heap called the string constant pool. When the same literal appears twice, both references point to the single pooled object. Creating a string with new String("Java") always creates a new object outside the pool.

```
String a = "Java";
String b = "Java";
String c = new String("Java");
a == b        // true: same pooled object
a == c        // false: different objects
a.equals(c)   // true: same characters
```

Use equals() to compare the contents of two strings; == only checks whether two references point to the same object.

4.2 Common String methods

Method	Returns
length()	Number of characters
charAt(i)	Character at index i
substring(b, e)	Characters from index b up to, but not including, e
indexOf(s)	First index of s, or -1
toUpperCase(), trim()	A new, changed String

4.3 StringBuilder and StringBuffer

StringBuilder is a mutable sequence of characters. Its append(), insert(), reverse() and delete() methods change the same object, so building a string in a loop does not create a new object on each pass. StringBuffer offers the same methods but is synchronized, which makes it thread safe and slower.

String concatenation is evaluated from left to right: once one operand is a String, + means concatenation, so "5" + 3 + 2 gives "532" while 3 + 2 + "5" gives "55".

5. Exception Handling

An exception is an event that disrupts the normal flow of a program at run time. Every exception class is a subclass of Throwable, which has two branches: Error, for serious problems that a program should not try to catch (for example OutOfMemoryError), and Exception, for conditions that a program can handle.

5.1 Checked and unchecked exceptions

Checked exceptions are checked by the compiler and must be either caught or declared with throws; unchecked exceptions are subclasses of RuntimeException. Examples of checked exceptions are IOException and SQLException. Examples of unchecked exceptions are NullPointerException, ArithmeticException and ArrayIndexOutOfBoundsException.

5.2 try, catch and finally

```
try {
    int result = 10 / 0;              // throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Clean-up code");
}
```

A try block can be followed by several catch blocks; they are checked in order, so a subclass exception must be caught before its superclass. The finally block always executes, whether or not an exception is thrown, unless the JVM exits first by calling System.exit().

5.3 throw and throws

The throw statement raises an exception object explicitly, for example throw new IllegalArgumentException("Age must be positive"). The throws clause in a method signature declares the checked exceptions that the method may pass to its caller.

5.4 User-defined exceptions

A custom checked exception extends Exception; a custom unchecked exception extends RuntimeException. The constructor usually passes a message to super(message).

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) { super(msg); }
}
```

6. Collections Framework

The Java Collections Framework is a set of interfaces and classes in java.util for storing and processing groups of objects. The main interfaces are List, Set, Queue and Map. Map is part of the framework but does not extend the Collection interface.

Interface	Ordering	Duplicates	Common classes
List	Insertion order, index based	Allowed	ArrayList, LinkedList, Vector
Set	None (HashSet), sorted (TreeSet)	Not allowed	HashSet, LinkedHashSet, TreeSet
Queue	FIFO or priority	Allowed	PriorityQueue, ArrayDeque
Map	None (HashMap), sorted keys (TreeMap)	Unique keys	HashMap, LinkedHashMap, TreeMap

6.1 ArrayList and LinkedList

ArrayList is backed by a resizable array, so get(i) is O(1) but inserting in the middle is O(n). LinkedList is a doubly linked list, so adding or removing at either end is O(1) but get(i) is O(n).

6.2 HashMap

A HashMap stores key-value pairs in an array of buckets. The bucket for a key is chosen from its hashCode(), and equals() is used to find the key inside the bucket. A HashMap allows one null key and any number of null values, does not maintain insertion order and is not synchronized.

> Resize threshold = capacity × load factor. Default capacity 16 and load factor 0.75 give a threshold of 12 entries.

When the number of entries exceeds capacity × load factor, the HashMap doubles its capacity and rehashes every entry.

6.3 Iterating over a collection

```
List<String> names = new ArrayList<>();
names.add("Ravi");
names.add("Meera");
for (String n : names) {
    System.out.println(n);
}
```

Modifying a collection while iterating over it with a for-each loop throws ConcurrentModificationException; use the Iterator's remove() method instead.

7. Multithreading

A thread is a lightweight unit of execution within a process. Threads of the same process share the heap but each has its own stack. Multithreading lets a program perform several tasks at the same time and keeps it responsive.

7.1 Creating threads

There are two ways to create a thread: extend the Thread class and override run(), or implement the Runnable interface and pass the object to a Thread constructor. Implementing Runnable is preferred because the class can still extend another class.

```
class Task implements Runnable {
    public void run() { System.out.println("Running"); }
}
Thread t = new Thread(new Task());
t.start();
```

Calling run() directly executes the method on the current thread; only start() creates a new thread of execution.

7.2 Thread life cycle

A thread moves through these states: New (created, not started), Runnable (ready or running), Blocked (waiting for a lock), Waiting or Timed Waiting (waiting for another thread or for a time limit, for example after sleep()), and Terminated (run() has finished).

7.3 Synchronization

When several threads update shared data at the same time, the result can depend on their timing; this is called a race condition. A synchronized method acquires the lock on the object, so only one thread at a time can execute any synchronized method of that object. A synchronized block locks only the statements that need protection.

7.4 Useful Thread methods

Method	Purpose
sleep(ms)	Pauses the current thread; it keeps any locks it holds
join()	Waits for another thread to finish
wait() / notify()	Releases the lock and waits / wakes a waiting thread; called inside synchronized code
setPriority(n)	Sets priority from 1 (MIN) to 10 (MAX); default is 5